Travel Counts — Análisis de flujo App Móvil vs Ruby BE

Fecha: 2026-04-10
Archivos analizados:

- App Android: `app/.../ActivitiesChofer/Viajes/AgregarViajes/ConteoViajes.java`
- PHP: `ttpn_php/Gasto_INSERT_TRAVEL_COUNTS.php`
- Ruby Model: `ttpngas/app/models/travel_count.rb`
- Ruby Controller: `ttpngas/app/controllers/api/v1/travel_counts_controller.rb`
- Funciones PG: `ttpngas/db/migrate/20260115172945_create_postgres_functions_cuadre_viajes.rb`
- Trigger PG: `ttpngas/db/migrate/20260310035106_create_before_insert_trigger_for_travel_counts.rb`

Contexto arquitectónico

Ruby BE siempre ha sido la fuente de verdad. Creó las tablas, triggers, funciones PostgreSQL (`buscar_booking`, `buscar_booking_id`, `buscar_nomina`). PHP era un proxy delgado sobre esa base de datos. La app Android consume PHP, que a su vez ejecuta las funciones que creó Ruby.

La migración correcta: **App Android → Rails API** (PHP desaparece, el módulo Acceso API lo sustituye).

---

1. Campos que envía la app Android al crear un travel_count

Campo	Valor	Origen	¿Hardcodeado?
`id`	max_id + 1	App calcula el siguiente autoincrement	⚠ Patrón peligroso
`employee_id`	`sEmployee_id`	Intent.getStringExtra (login)	Dinámico
`vehicle_id`	`sVehicle_id`	Intent.getStringExtra (login)	Dinámico
`client_branch_office_id`	spinner UI	Selección del usuario	Dinámico
`ttpn_service_type_id`	botón 1 o 2	Botones Entrada/Salida	Dinámico
`fecha`	DatePicker	Usuario	Dinámico
`hora`	TimePicker ± ajuste	Usuario, con lógica de +15 min	⚠ Modificado por app
`ttpn_foreign_destiny_id`	spinner / override	**HARDCODEADO con overrides** (ver §2)	🔴 Crítico
`costo`	calculado	Parámetro del formulario	Dinámico

Campo	Valor	Origen	¿Hardcodeado?
`status`	`"true"`	**Siempre true**	🔴 Hardcodeado
`created_at`	timestamp sistema	Automático	Dinámico
`updated_at`	= `created_at`	**Igual a created_at en creación**	⚠ Nunca se actualiza

Campos que NO envía la app (los calcula el backend)

Campo	Quién lo calcula	Cómo
`viaje_encontrado`	Trigger PG	`buscar_booking(...)` → BOOLEAN
`ttpn_booking_id`	Trigger PG	`buscar_booking_id(...)` → BIGINT
`payroll_id`	PHP llama `buscar_nomina()`	**NO está en el trigger de Ruby**
`clv_servicio`	Trigger PG	Formato: `client_id-fecha-HH:MM:SS-type-dest-vehicle`
`created_by_id`	Trigger PG	Default a 1 si NULL
`updated_by_id`	Trigger PG	Default a 1 si NULL
`desactivacion`	PHP	NULL siempre
`comentario`	PHP	NULL siempre
`requiere_autorizacion`	PHP/Trigger	`false` siempre
`errorcode`	Trigger PG	`true` si `client_branch_office_id < 0`

2. Lógica hardcodeada en la app que debe migrar al BE

2a. Override de `ttpn_foreign_destiny_id` 🔴

La app tiene esta lógica en `ConteoViajes.java` líneas ~1249-1283:

```
// Si el spinner devuelve -1, forzar destino 1 (Chihuahua)
if (sTtpn_foreign_destiny_id.equals("-1")) {
    sTtpn_foreign_destiny_id = "1";
}

// Clientes especiales → forzar destino 11 (Especial)
if (sClient_branch_office_id.equals("263") ||
    sClient_branch_office_id.equals("262") ||
    sClient_branch_office_id.equals("261") ||
    sClient_branch_office_id.equals("196")) {
```

```
    sTtpn_foreign_destiny_id = "11";
}
```

Problema: Esta regla de negocio (sucursales Emerzon, Termotec, Safran, Zodiac → destino Especial) está hardcodeada en el APK. Si cambian los clientes o se agregan más, hay que recompilar la app.

Solución en Rails: Agregar campo `default_ttpn_foreign_destiny_id` a `client_branch_offices`, o una tabla de reglas. El API resuelve el destino correcto en el backend basándose en la sucursal.

```
# En travel_counts_controller.rb, antes de crear:
if params[:ttpn_foreign_destiny_id].blank?
  branch = ClientBranchOffice.find(params[:client_branch_office_id])
  params[:ttpn_foreign_destiny_id] =
branch.default_ttpn_foreign_destiny_id ||
branch.client.default_ttpn_foreign_destiny_id
end
```

2b. Ajuste de +15 minutos en hora ⚠

```
// Si la diferencia con el horario programado es ≤ 14 min, ajusta +15 min
if (timeDiff <= 14) {
    hora = hora + 15_minutos;
}
```

Problema: Lógica de detección de viajes dobles está solo en el cliente. Si alguien usa la API directamente, no se aplica.

Solución: Mover validación al modelo Ruby o al controller.

2c. `status` siempre `true`

La app nunca crea un travel_count inactivo. El campo no tiene utilidad real desde la app. El Rails API debería defaultear a `true` si no se envía.

---

3. SQL que ejecuta el PHP

```
INSERT INTO travel_counts VALUES(
  :id,
  :employee_id,
  :client_branch_office_id,
  :ttpn_service_type_id,
  ':fecha',
  time ':hora',
  :ttpn_foreign_destiny_id,
  :costo,
```

```
    :status,
    ':created_at',
    ':updated_at',
    :vehicle_id,
    -- viaje_encontrado: llama función PostgreSQL
    COALESCE((SELECT buscar_booking(
      :vehicle_id, :employee_id, :ttpn_service_type_id,
      (SELECT client_id FROM client_branch_offices WHERE id =
  :client_branch_office_id),
      :ttpn_foreign_destiny_id,
      (date ':fecha' + time ':hora'),
      ((date ':fecha' + time ':hora') + '15 minutes'::INTERVAL)
    )), false),
    NULL,  -- desactivacion
    NULL,  -- comentario
    -- ttpn_booking_id: llama función PostgreSQL
    COALESCE((SELECT buscar_booking_id(
      :vehicle_id, :employee_id, :ttpn_service_type_id,
      (SELECT client_id FROM client_branch_offices WHERE id =
  :client_branch_office_id),
      :ttpn_foreign_destiny_id,
      (date ':fecha' + time ':hora'),
      ((date ':fecha' + time ':hora') + '15 minutes'::INTERVAL)
    )), NULL),
    0,                        -- errorcode
    (SELECT buscar_nomina()),  -- payroll_id ← FALTA EN RUBY TRIGGER
    false                     -- requiere_autorizacion
  )
```

Post-INSERT que ejecuta PHP (falta en Ruby):

```
UPDATE discrepancies
SET status = false
WHERE kpi = 'pnc'
  AND record_id = (
    SELECT ttpn_booking_id FROM travel_counts WHERE id = :id
  )
```

---

4. Estado del Trigger Ruby vs lógica PHP

El trigger PG de Ruby (`sp_tctb_insert`) hace:

Lógica	¿Está en trigger Ruby?	¿Está en PHP?
Generar `clv_servicio`	✅	No (solo trigger)
Calcular `viaje_encontrado` via `buscar_booking()`	✅	✅

Lógica	¿Está en trigger Ruby?	¿Está en PHP?
Calcular `ttpn_booking_id` via `buscar_booking_id()`	✅	✅
Setear `created_by_id = 1` si NULL	✅	No
Setear `errorcode` si branch < 0	✅	No
Calcular `payroll_id` via `buscar_nomina()`	🔴 **FALTA**	✅
Update `discrepancies` post-insert	🔴 **FALTA**	✅
Override `ttpn_foreign_destiny_id` por sucursal	🔴 **FALTA**	En app cliente

5. Gaps específicos a resolver en Ruby

Gap 1 — `buscar_nomina()` no se llama en el trigger 🔴

Estado: `buscar_nomina()` **ya existe** en Supabase (confirmado 2026-04-10 — visible en explorador de funciones).
El trigger `sp_tctb_insert` simplemente no la invoca.

Impacto: El `payroll_id` nunca se asigna automáticamente al crear desde Rails API.
Los registros quedan sin asociar a nómina → falla el cuadre de nómina.

Fix aplicado: Nueva migración `20260410000002_add_buscar_nomina_to_travel_counts_trigger.rb`
Agrega al trigger antes del `RETURN NEW`:

```
IF NEW.payroll_id IS NULL THEN
  NEW.payroll_id := (SELECT buscar_nomina());
END IF;
```

Correr en desarrollo/staging:

```
bundle exec rails db:migrate
# o en Docker:
docker compose exec api bundle exec rails db:migrate
```

Gap 2 — Discrepancias no se resuelven post-insert 🔴

Impacto: Cuando un travel_count hace match con un booking, la discrepancia `pnc` queda como no resuelta indefinidamente.

Fix: Agregar callback `after_create` en `TravelCount`:

```
after_create :resolver_discrepancia_pnc

def resolver_discrepancia_pnc
  return unless ttpn_booking_id.present?
  Discrepancy.where(kpi: 'pnc', record_id: ttpn_booking_id, status: true)
             .update_all(status: false)
end
```

Gap 3 — Override de destino por sucursal no está en backend 🟡

Impacto: Si la app se sustituye por el API (Acceso API), el destino correcto no se calcula.

Fix opción A — campo en `client_branch_offices`:

```
rails g migration AddDefaultForeignDestinyToClientBranchOffices \
  default_ttpn_foreign_destiny_id:integer
```

Luego en el controller:

```
def resolve_foreign_destiny
  return if params[:ttpn_foreign_destiny_id].present?
  branch = ClientBranchOffice.find(params[:client_branch_office_id])
  params[:ttpn_foreign_destiny_id] =
branch.default_ttpn_foreign_destiny_id ||
                                      TtpnForeignDestiny.find_by(nombre:
'Chihuahua')&.id
end
```

Fix opción B — setar las 4 sucursales existentes directamente en DB:

```
UPDATE client_branch_offices
SET default_ttpn_foreign_destiny_id = 11
WHERE id IN (263, 262, 261, 196);
```

Gap 4 — `created_by_id` siempre queda en 1 🟡

El trigger defaultea a `1` cuando no viene el usuario. Desde Rails API el `current_user` está disponible.

Fix: En el controller pasar `created_by_id` y `updated_by_id`:

```
def create
  @travel_count = TravelCount.new(travel_count_params)
  @travel_count.created_by_id = current_user.id
  @travel_count.updated_by_id = current_user.id
```

```
  # ...
end
```

Gap 5 — ID generado en el cliente 🟡

La app Android hace GET max_id + 1 antes de insertar. Con Rails API esto no aplica — Rails usa autoincrement de PostgreSQL.

Fix en la app: No enviar `id` en el body. Leer el `id` de la respuesta del POST.
Fix en el controller: Ignorar cualquier `id` que venga en params (Rails ya lo hace por defecto).

---

6. Campos faltantes verificar en schema

Verificar que `travel_counts` en `db/schema.rb` tenga:

```
# Campos que usa la app y el PHP
t.integer  "employee_id"
t.integer  "client_branch_office_id"
t.integer  "ttpn_service_type_id"
t.date     "fecha"
t.time     "hora"
t.integer  "ttpn_foreign_destiny_id"
t.decimal  "costo"
t.boolean  "status"
t.integer  "vehicle_id"
t.boolean  "viaje_encontrado"       # trigger
t.string   "desactivacion"
t.string   "comentario"
t.bigint   "ttpn_booking_id"        # trigger
t.integer  "errorcode"              # OJO: ¿INTEGER o BOOLEAN?
t.bigint   "payroll_id"             # ← asignar via buscar_nomina()
t.boolean  "requiere_autorizacion"
t.string   "clv_servicio"           # trigger
t.bigint   "created_by_id"         # trigger default 1
t.bigint   "updated_by_id"         # trigger default 1
```

---

7. Plan de acción priorizado

🔴 Inmediato — sin esto el cuadre falla

1. **Crear/verificar función** `buscar_nomina()` en PostgreSQL y agregar llamada al trigger
2. **Agregar** `after_create :resolver_discrepancia_pnc` en el modelo `TravelCount`
3. **Verificar campo** `errorcode` — ¿INTEGER o BOOLEAN? Unificar en schema

🟡 Antes de deprecar PHP

4. **Agregar `default_ttpn_foreign_destiny_id`** a `client_branch_offices` y setar las 4 sucursales especiales
5. **Pasar `created_by_id` / `updated_by_id`** desde el controller con `current_user.id`
6. **Mover lógica +15 min** al model o controller como validación

🔵 Cuando la app actualice

7. **Eliminar fetch de max_id** de la app — usar autoincrement de Rails
8. **Actualizar `status` default** — el controller puede defaultear a `true` si no viene
9. **Respuesta del POST** incluir todos los campos calculados por trigger para que la app los use

---

8. Endpoint Rails recomendado para la app

```
POST /api/v1/travel_counts
Headers: Authorization: Bearer <jwt_token>
Body:
{
  "travel_count": {
    "employee_id": 10,
    "vehicle_id": 5,
    "client_branch_office_id": 42,
    "ttpn_service_type_id": 1,
    "fecha": "2026-04-10",
    "hora": "08:30:00",
    "costo": 250.00
    // ttpn_foreign_destiny_id: opcional, el BE lo resuelve si falta
    // status: opcional, default true
    // id: NO enviar, Rails usa autoincrement
  }
}

Response 201:
{
  "id": 1235,
  "clv_servicio": "42-2026-04-10-08:30:00-1-2-5",
  "viaje_encontrado": true,
  "ttpn_booking_id": 890,
  "payroll_id": 12,
  "employee_id": 10,
  "vehicle_id": 5,
  ...
}
```